MESA Project Charter

Charter Summary

MESA is a real-time, shared operations console for data-center teams. Every warning gets an owner and a deadline, every shift change requires the next operator to explicitly accept each open issue, and every alert shows how long until it gets worse. It is built on SpacetimeDB, so all operators and AI agents work from one live, server-enforced state.

Mission. No alert left unowned: catch problems early, hand them over cleanly, and give operators the right information at the moment they need it.

Item	Detail
Event	HackUTD 2026: Zero Day, UT Dallas
Tracks entered	NMC² ("Keep the World Running") and Spacetime (main track)
Build window	Hacking started 12:00 pm Sat Nov 7; initial Devpost entry due 11:00 pm Sat; final submission due 12:00 pm Sun Nov 8 (team target 11:30 am)
Judging	1:00–4:00 pm Sun Nov 8
Team	4 people: 2 full-stack, 2 back-end; 3 AI/ML, 1 cybersecurity
Name rationale	MESA (meaning or expansion to be added by the team)
Hard requirement	Public GitHub repository (required to win the Spacetime prize)

Problem and Business Use Cases

Data centers rarely fail from a single event; they fail when a small, visible warning is missed, handed over badly, or understood too late. NMC²'s challenge names these three failure modes directly: a missed warning, an unclear handoff, and the wrong information at the wrong time. Today, most operations teams spread these across separate tools: a monitoring system that fires alerts, a chat channel where people discuss them, and a shift log written at the end of the night. The gaps between those tools are where incidents grow.

Primary use case: the night-shift network or facilities operations center. An operator watches power, cooling and compute health across a data hall. A cooling unit starts to fail at 5:40 am, twenty minutes before shift change. MESA gives the alert an owner, predicts when rack temperatures cross the safe limit, attaches the runbook step, and refuses to let the shift end until the incoming operator accepts the issue.

Additional business use cases.

Customer	Situation	What MESA provides

Customer	Situation	What MESA provides
HPC and GPU cloud operators (NMC²'s own business)	Long-running research jobs on dense GPU clusters, where a thermal or power event kills days of compute	Early warning with time-to-threshold, and owned escalation before jobs fail
Colocation providers	Many tenants, contractual uptime commitments, rotating on-site staff	A verifiable record of who knew what and when, useful for customer reporting
Enterprise IT and cloud operations teams	Follow-the-sun support passing incidents across time zones	Explicit cross-region handoff instead of a chat message that may be missed
Other 24/7 operations (energy field operations, utilities, telecom network centers)	The same shift-change risk in other critical infrastructure	The same model with different telemetry; a credible path beyond data centers

Value to the business. Fewer incidents escalating into outages, faster response to the ones that do, and an audit trail that shortens post-incident reviews. We will state these as qualitative benefits with demo evidence rather than invent dollar figures.

Track Alignment

MESA is one product entered in both tracks: NMC² supplies the problem, and Spacetime supplies the technology that makes the solution work. Each judging panel should be able to find its criteria in the demo within the first minute.

NMC²: "Keep the World Running"

What NMC² asks for	How MESA answers it
Prevent a missed warning	Every alert has an owner and an acknowledgment deadline; unacknowledged alerts escalate automatically to the next person on call
Prevent an unclear handoff	Shift change is a locked process: each open item must be explicitly accepted by the incoming operator, with an AI-drafted handoff brief
Avoid the wrong information at the wrong time	Alerts show a predicted time until the situation worsens and the relevant runbook step, not raw metric streams
Help teams "stay one step ahead"	Trend-based prediction raises an alert before a hard limit is crossed, not after

What NMC² asks for	How MESA answers it
Fit NMC²'s world	Telemetry models a dense GPU and high-performance compute hall, NMC²'s own business, including power and cooling relationships

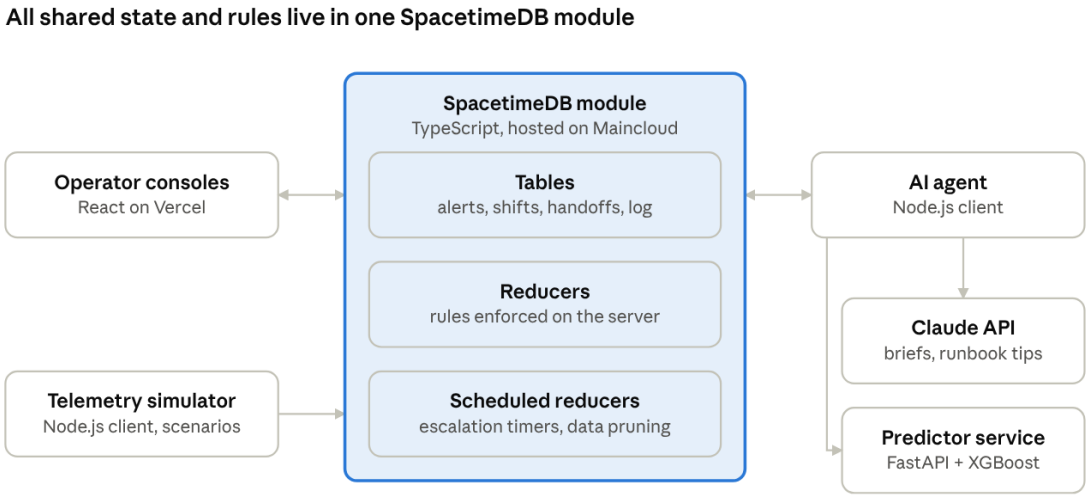
Spacetime: Main Track

Spacetime requirement or strength	How MESA uses it
Spacetime must power the app's primary functionality, "not just chat or player movement"	All core state lives in SpacetimeDB: telemetry, alerts, ownership, acknowledgments, shifts, handoff items and the event log. Without it the product does not function
Server-side logic inside the database (reducers)	Business rules are enforced on the server: an alert cannot be closed without an owner, and a shift cannot end with unaccepted items
Real-time sync to every client (subscriptions)	Every operator screen, the AI agent and the simulator see the same state within moments, demonstrated live on two laptops
Built-in client identity	Each connected operator has a SpacetimeDB identity; a roles table controls who may acknowledge, escalate or end a shift
Scheduled reducers	Escalation timers run inside the database, so escalation still happens if every browser is closed
Public source code on GitHub	Repository made public from the start

Why the pairing is credible. Operations software is fundamentally a shared, real-time state problem. Most teams in the Spacetime track will build games; MESA shows Spacetime solving a serious infrastructure problem, while NMC²'s judges see a product shaped around their exact wording.

Stack and Architecture

MESA runs almost entirely in TypeScript around a single SpacetimeDB module, which is fully supported for server code according to the SpacetimeDB docs. The only Python is the prediction service, where our ML strength lives.



MESA architecture · three clients, one SpacetimeDB module, two outside services

Operators, the telemetry simulator and the AI agent are all SpacetimeDB clients: they subscribe to tables and change data only by calling reducers. The agent is the only component that talks to outside services.

Layer	Technology	Role and reason
Database and server logic	SpacetimeDB TypeScript module, published free to Maincloud (local `spacetime start` for development)	Holds all state, enforces rules in reducers, runs escalation timers as scheduled reducers
Operator console	React + TypeScript with the `spacetimedb/react` integration, on Vercel	Live subscriptions; no REST layer needed for core state
Telemetry simulator	Node.js TypeScript client using `spacetimedb` SDK, on Railway in Docker	Streams synthetic rack, power and cooling readings; triggers scripted failure scenarios
AI agent	Node.js TypeScript client on Railway	Watches new alerts and shift changes; writes predictions, runbook hints and handoff briefs back through reducers
Prediction service	Python FastAPI with scikit-learn and XGBoost, on Railway in Docker	Estimates time until a reading crosses its limit
Language model	Anthropic Claude	Handoff briefs, runbook suggestions, plain-English alert summaries

Layer	Technology	Role and reason
Optional voice	ElevenLabs	Spoken escalation for an unacknowledged critical alert
Delivery	GitHub (public), GitHub Actions for type checks and module build	Required public source for the Spacetime prize

Data model (first draft)

Table	Key fields	Notes
`operator`	identity, name, role (operator, lead, observer), on_shift	Identity comes from the SpacetimeDB connection
`asset`	asset_id, type (rack, cooling unit, power unit), zone, limits	Seeded at startup
`reading`	asset_id, metric, value, timestamp	Keep a rolling window; a scheduled reducer prunes old rows
`alert`	alert_id, asset_id, severity, status, owner, ack_deadline, predicted_breach_at, runbook_hint	Status: open, acknowledged, mitigating, resolved
`shift`	shift_id, outgoing, incoming, started_at, status	Status: active, handing over, closed
`handoff_item`	shift_id, alert_id, note, accepted_by, accepted_at	One per open alert at shift change
`event_log`	seq, actor, action, target, timestamp, prev_hash, hash	Append-only and hash-chained for tamper evidence
`escalation_job`	scheduled_at, alert_id	Scheduled table that triggers the escalation reducer

Core reducers. `ingest_reading`, `raise_alert`, `acknowledge_alert`, `assign_alert`, `resolve_alert` (rejects alerts without an owner), `attach_insight` (agent only), `begin_handoff`, `accept_handoff_item`, `end_shift` (rejects while any item is unaccepted), `escalate` (scheduled), and `prune_readings` (scheduled). Every reducer appends to `event_log`.

Repository Structure

Use one shared repository organized as a monorepo, with one top-level folder per owner so four people can push all night without colliding. The SpacetimeDB module and its generated bindings are the shared contract, so they get extra review.

```
mesa/
├─ module/                  SpacetimeDB TypeScript module (back-end 1)
│  └─ src/
│     ├─ index.ts           schema export
│     ├─ tables/            one file per table
│     ├─ reducers/          alerts.ts, shifts.ts, insights.ts
│     ├─ scheduled/         escalate.ts, prune.ts
│     └─ honeypot/          decoy reducers, canary rows, threat events
├─ packages/
│  └─ bindings/             generated client bindings (never edited by hand)
├─ apps/
│  └─ console/              React + Vite operator console (full-stack 1)
├─ services/
│  ├─ simulator/            Node telemetry simulator and scenarios (back-end 2)
│  ├─ agent/                Node AI agent: Claude and predictor calls (full-stack 2)
│  └─ predictor/            Python FastAPI + XGBoost, own requirements.txt and
Dockerfile
├─ security/
│  ├─ decoy-web/            decoy admin portal deployed separately (security)
│  └─ attack-scripts/       scripted intrusion used in the demo (security)
├─ scripts/                 generate-bindings.sh, publish.sh, run-demo.sh
├─ docs/                    charter, architecture, demo script
├─ .github/
│  ├─ workflows/ci.yml      type checks, module build, Python lint
│  └─ CODEOWNERS
├─ .env.example
└─ README.md                one-command local setup for judges
```

Repository rules.

- **npm workspaces** link `apps/*`, `services/*` (except the Python predictor) and `packages/*`, so every TypeScript client imports bindings from one place.
- **One script regenerates bindings** (`spacetime generate --lang typescript` into `packages/bindings`). Run it after every schema change and commit the result in the same pull request.
- **CODEOWNERS** requires the module owner's review for `module/` and `packages/bindings/`, and the security specialist's review for `security/` and `module/src/honeypot/`.
- **Protect `main`:** no direct pushes, CI must pass, short-lived branches named `owner/feature`.
- **Secrets never enter the repo.** The repo must be public for the Spacetime prize, so commit only `.env.example`; keep real keys in Railway and Vercel environment variables, and turn on GitHub secret scanning with push protection.
- **README first:** what MESA is, both tracks, the architecture diagram, and how to run it locally in under five minutes.

Security Layer: Honeypot

The honeypot turns MESA's own attack surface into an early-warning system. Anyone probing the system gets caught, and the catch becomes a critical alert that flows through MESA's normal ownership, escalation and handoff process. This strengthens all three judging stories: NMC² (data centers face intrusions as well as hardware failures), Spacetime (it uses reducers and client identity in a non-game way), and the Zero Day theme.

The design takes advantage of a SpacetimeDB property: public reducers can be called by any connected client unless the module enforces authorization. Real reducers check roles; decoy reducers deliberately look privileged and exist only to be found.

Component	How it works	What a hit means
Decoy reducers	Tempting names such as `admin_override_alarm` or `export_operator_tokens`. They change nothing real; they record the caller's identity in a `threat_event` table	No legitimate client ever calls them, so any call is hostile
Canary records	A decoy table holds fake "maintenance credentials." They work nowhere, but any attempt to use them is logged	Someone read data they should not have and tried to use it
Decoy admin portal	A fake "Facility Management Admin" login page deployed separately on Railway, isolated from real services	Scanning or credential-stuffing against the organization
Quarantine	Identities that touch any decoy are flagged; every real reducer rejects flagged identities	Containment within seconds

Pipeline. A decoy hit writes a `threat_event` row, which raises a critical security alert in the normal `alert` table. Escalation and handoff rules then apply. The AI agent drafts an attacker timeline from the event log.

Safety rules. Decoys never modify real state; the portal stores only hashes of anything typed, never raw passwords; everything runs on our own infrastructure; the portal is rate-limited; and the repository labels these components clearly as a honeypot.

Demo addition (20 s). The security specialist runs the attack script from a third laptop. The intrusion alert appears on both operator screens, escalates, and the attacker's identity is quarantined.

Scheduling. Treat the honeypot as a Should-priority feature owned by the security specialist, built in parallel with the core work from about 6 pm. It must not delay the 11:00 pm handoff gate.

Spacetime Credits and Deployment

The teammate with Spacetime credits should own the production database on Maincloud and act as release manager, while everyone else develops locally for free. This keeps credits for the shared demo environment and avoids sharing account logins.

- **Production database.** The credit holder creates the `mesa` database on Maincloud and is the only person who publishes to it, from `main`, at each checkpoint gate. If the credit holder is not the module owner, the module owner merges and the credit holder publishes.
- **Development.** Everyone else runs `spacetime start` locally and publishes the module to their own machine. Clients switch between local and production with one environment variable.

- **Conserve credits.** Throttle the simulator (for example, one reading per asset every 2 seconds), subscribe only to the latest reading per asset, prune old readings with the scheduled reducer, and stop the simulator when no one is testing.
- **Watch usage.** The database owner can see utilization on the Maincloud database dashboard; check it at each gate.
- **Ask the Spacetime mentors early:** how far the credits will stretch for an overnight simulator, whether other teammates can be given publish rights to the same database, and whether CI can publish with a token.
- **Fallback.** Keep a local SpacetimeDB running on the demo laptop with the same module, in case venue Wi-Fi or credits run out during judging.

Development Procedure

Build the shared state first, then the rules, then the intelligence; each layer must work end to end before the next one starts. The module schema is the contract between all four teammates, so it is agreed and published before anyone builds on it.

Build order

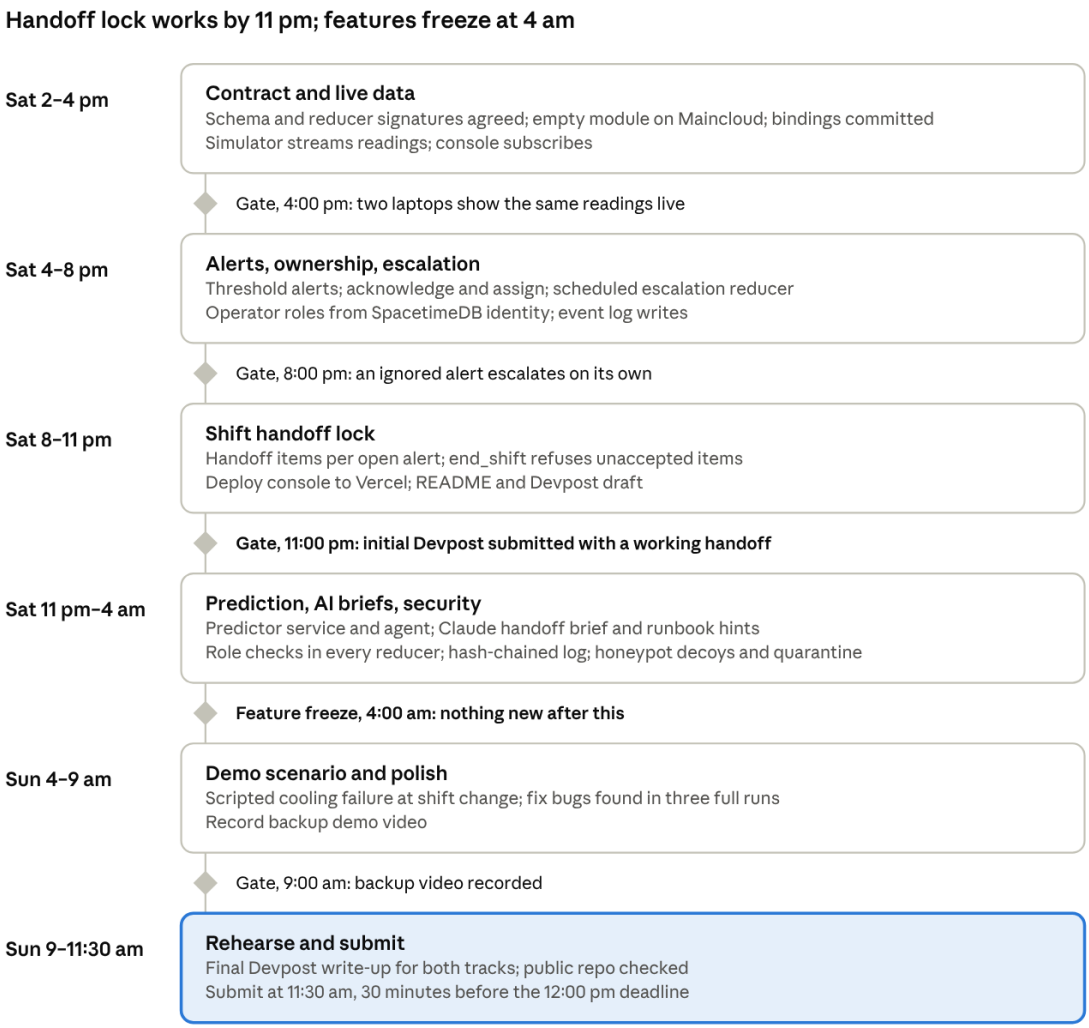
1. **Contract (first hour).** Agree on table names, fields and reducer signatures as written in the Data model above. Publish an empty module to Maincloud and generate TypeScript client bindings with `spacetime generate`. Commit the bindings so every client imports the same types.
2. **Live data path.** The simulator streams readings through `ingest_reading`. The console subscribes and shows them. Proof: two laptops show the same values at the same moment.
3. **Alerts and ownership.** Threshold rules inside `ingest_reading` raise alerts. Operators acknowledge and assign them. The `escalate` scheduled reducer reassigns any alert not acknowledged before its deadline.
4. **Shift handoff.** `begin_handoff` creates one handoff item per open alert; `end_shift` refuses while any item is unaccepted. This is the signature feature, so it gets polish before anything optional.
5. **Prediction.** The agent sends a rolling window to the predictor service and writes `predicted_breach_at` back through `attach_insight`. Start with a linear trend; switch to XGBoost only if the baseline works.
6. **AI briefs and runbook hints.** The agent drafts the handoff brief from `event_log` and a one-line runbook hint per alert using Claude.
7. **Security hardening.** Role checks in every reducer, the hash-chained `event_log` with a verify button, and secrets kept in environment variables.
8. **Demo scenario and polish.** A scripted cooling failure during shift change, triggered on cue by the simulator.

Working rules

- **One owner per layer:** module and rules (back-end 1), simulator and predictor (back-end 2), console (full-stack 1), agent and deployment (full-stack 2). The security specialist also owns step 7.

- **Schema changes go through the module owner** and are announced in the team chat with regenerated bindings, because every client breaks on a mismatch.
- **Main branch always deploys.** Work in short-lived branches; GitHub Actions runs type checks and the module build on every pull request.
- **No fake functionality in the demo.** Everything shown must run live; anything unfinished is described as a roadmap item, not mocked.
- **Commit and push at every checkpoint**, so the 11:00 pm initial Devpost entry and the final submission point to a working state.

Checkpoints



Checkpoints · 6 phases, 5 gates, Sat 2 pm to Sun 11:30 am

A missed gate means cutting scope, not borrowing time from the next phase: drop XGBoost for the linear trend, drop voice, then drop the AI brief. The handoff lock and live sync are never cut, because they are what both tracks judge.

Feature Options

Five features are required for the demo; the rest are options to add, in order, only after the previous gate passes. Effort is rough team-hours.

Feature	Priority	Strengthens	Effort	Notes
Live shared state across all screens	Must	Spacetime	2 h	The foundation; proven with two laptops
Alert ownership with acknowledgment deadlines	Must	NMC² (missed warning)	3 h	Owner and deadline required on every alert
Automatic escalation by scheduled reducer	Must	Both	2 h	Runs inside the database even if every browser is closed
Shift handoff lock	Must	NMC² (unclear handoff), Spacetime	3 h	`end_shift` refuses while any item is unaccepted; the signature feature
Scripted failure scenarios in the simulator	Must	Demo	2 h	Cooling-unit failure, power-unit overload, sensor dropout
Time-to-breach prediction per alert	Should	NMC² (one step ahead)	3 h	Linear trend first; XGBoost upgrade
AI handoff brief from the event log	Should	NMC² (right information)	2 h	Claude summarizes the shift in five lines; the incoming operator reads it before accepting
Runbook hint per alert	Should	NMC²	1.5 h	One recommended next step, with a link to the full runbook text
Role-based permissions from SpacetimeDB identity	Should	Both, security	2 h	Only leads can end a shift; the agent can only attach insights
Hash-chained, tamper-evident event log with a Verify button	Should	Security, post-incident review	1.5 h	Owned by our security specialist
Cascade awareness	Could	NMC²	3 h	Shows that a failing cooling unit affects specific racks, linked by zone
Spoken escalation (ElevenLabs)	Could	Demo impact	1 h	An unacknowledged critical alert calls out the operator by name

Feature	Priority	Strengthens	Effort	Notes
Replay mode	Could	Spacetime, post-incident review	3 h	Re-run the shift from `event_log` at faster speed
Post-incident report generator	Could	NMC²	2 h	Claude drafts a timeline report from the event log
Mobile on-call view	Could	NMC²	2 h	The escalated person acknowledges from a phone

Out of scope this weekend: real monitoring integrations (Prometheus, vendor building-management systems), paging providers, and multi-site support. List them as the roadmap in the pitch.

UI/UX Design and Enhancement Options

The console is designed for a tired operator at 5 am, not for a wall display. Each screen answers one question: what needs me, what is about to get worse, and what am I handing over.

Core screens

1. **My Queue (default).** Only alerts owned by me or unowned in my zone, sorted by predicted time to breach. Each row shows the asset, a plain-English status ("Rack B-12 inlet rising, crosses 32°C in about 14 min"), the owner, a countdown to the acknowledgment deadline, and one primary button.
2. **Alert detail.** A small inline sparkline, the runbook hint, related assets, and the alert's own event history. Actions: Acknowledge, Assign, Mitigating, Resolve.
3. **Shift Handoff.** A checklist of open items. Each item shows the AI note and the outgoing operator's note; the incoming operator must tap Accept on each one. The End Shift button stays disabled, with the count of unaccepted items shown on it, until all are accepted.
4. **Floor view (secondary).** A simple grid of the data hall with rack tiles shaded by status, so a newcomer can orient themselves. It is a map, not the home screen.

Enhancement options

Option	What it does	Why it helps
Presence avatars	Shows who is viewing each alert right now, using SpacetimeDB subscriptions	Prevents two operators working the same issue blindly; makes real-time sync visible to Spacetime judges
Countdown chips	Deadline and predicted-breach timers that change color as time runs down	Urgency at a glance without reading numbers
Keyboard-first actions	A to acknowledge, E to escalate, H to start handoff, J/K to move	Fast under stress; matches how operators use terminals

Option	What it does	Why it helps
Quiet mode at night	Dark theme, low-contrast idle rows, high contrast only for alerts needing action	Reduces fatigue; draws the eye only where needed
Plain-language status lines	Every alert includes a one-sentence summary instead of a metric name	Delivers the "right information at the right time" NMC² asks for
Handoff progress bar	"4 of 6 items accepted" at the top of the handoff screen	Makes the lock understandable, not frustrating
Undo window	A five-second undo after Resolve or Accept	Prevents costly mis-taps under stress
Live activity rail	A narrow side feed of the latest events from the event log	Shared situational awareness; shows real-time behavior in the demo
Accessibility	Color plus icon for every status, readable font sizes, screen-reader labels	Status never depends on color alone
Mobile acknowledgment	A compact page for the escalated person to acknowledge from a phone	Closes the loop when someone is away from the desk

Design principle for judges. Charts are supporting evidence inside an alert, never the product. The product is the decision and the ownership.

Demo Script, Risks and Open Questions

Demo script (3 minutes, two laptops)

1. **Problem (20 s).** Read NMC²'s three failure modes aloud: missed warning, unclear handoff, wrong information at the wrong time.
2. **Live shared state (25 s).** Two operators on two laptops see the same hall and the same readings update together. Mention that all of it lives in one SpacetimeDB module.
3. **The failure (45 s).** Trigger the cooling-unit failure at 5:40 am in the simulated clock. An alert appears on both screens with a predicted breach countdown and a runbook hint.
4. **The missed warning that isn't (25 s).** Operator A ignores it; the scheduled reducer escalates it to Operator B when the deadline passes.
5. **The handoff lock (40 s).** Operator A tries to end the shift; the button refuses with "1 item unaccepted." Operator B reads the AI brief and accepts. The shift closes.
6. **Proof and close (25 s).** Open the event log, run Verify, and show the full timeline. Close with the business case and roadmap.

Risks and mitigations

Risk	Mitigation
Team is new to SpacetimeDB	Start from the official TypeScript quickstart; ask Spacetime mentors early; keep the schema small
Schema changes break clients overnight	Single module owner; regenerate and commit bindings with every change
Venue Wi-Fi drops during judging	Run a local SpacetimeDB (`spacetime start`) on the demo laptop as a fallback, plus the backup video
Prediction looks fake on synthetic data	Show the trend line and a stated uncertainty; describe it honestly as a baseline
Too many readings slow subscriptions	Subscribe to latest readings per asset only; prune with a scheduled reducer
The agent takes an unsafe action	The agent may only call `attach_insight`; role checks in reducers block everything else

Open questions to settle now

- ☐ Confirm with Spacetime mentors that a TypeScript module on Maincloud meets their main-track expectations.
- ☐ Ask at NMC²'s 4:20 pm workshop which telemetry signals their operators watch most (cooling, power, GPU health) and use their wording.
- ☐ Decide who presents to which judges, since we are judged by both tracks during the 1:00–4:00 pm window.
- ☐ Choose the data-hall size for the simulator (suggested: 4 rows × 6 racks, 2 cooling units, 2 power units).